eForte New — Final Static Code Analysis Report

Field	Value
Project	eForte New (`eforte-new`)
Tool	SonarQube Community **26.10.0** (local)
Quality profile	Sonar way **comprehensive** (TypeScript, JavaScript, CSS, Docker, secrets/text)
Scope	`apps/web` , `apps/api`
Analysis period	2026-10-29 → 2026-10-30
Report type	Final summary (Rounds 1–7)
Audience	Engineering / QA / leadership

---

1. Executive summary

Static analysis was run against the Next.js web app and NestJS API with SonarQube comprehensive rules. Findings were remediated iteratively across seven analysis rounds.

Metric	Baseline (Round 1)	Final (Round 7)
Open issues	**447**	**0**
Bugs	**13**	**0**
Vulnerabilities	**2**	**0**
Code smells	**432**	**0**
Security hotspots	**0**	**0**
Reliability rating	D (4.0)	**A (1.0)**
Security rating	C (3.0)	**A (1.0)**
Maintainability rating	A (1.0)	**A (1.0)**
Technical debt (issues)	~2785 min	**0 min**
Lines of code (approx.)	~59k	~57k

Outcome: No open bugs, vulnerabilities, code smells, or security hotspots remain.

Quality gate caveat: The SonarQube quality gate may still show **ERROR** solely because **coverage is 0%** (gate expects ≥80% on new code). No unit-test / coverage sensor is configured for this analysis. That gate condition is independent of the cleared issue backlog.

---

2. Scope & methodology

Scope

- **Web:** Next.js App Router application under `apps/web`
- **API:** NestJS service under `apps/api` (including mail, careers, Docker image)

Process

1. Stand up local SonarQube (Community) with comprehensive quality profiles
2. Run Sonar Scanner against configured sources
3. Export measures + unresolved issues
4. Fix findings by priority (vulnerabilities → bugs → high-volume smells)
5. Rescan and document each round until issues reached zero

Intentional rule exception

Rule	File	Rationale
`typescript:S3923`	`Navbar.tsx`	Active nav link text stays white by design; active state is shown via underline, not a different text color

---

3. Baseline findings (Round 1)

Type	Count
Bugs	13
Vulnerabilities	2
Code smells	432
Total	**447**

Severity mix (baseline)

Severity	Count
CRITICAL	16
MAJOR	114
MINOR	317

Security (baseline)

ID	Rule	Location	Summary
V1	`typescript:S2245`	`apps/api/src/mail/mail.service.ts`	Unsafe use of `Math.random` for an entity / correlation id
V2	`docker:S6471`	`apps/api/Dockerfile`	Container process running as root by default

No committed secrets were flagged by the secrets/text sensors.

Representative bug themes included unsafe / unstable sorting, unhandled bootstrap promises, and accessibility patterns on interactive overlays (navbar backdrops).

---

4. Remediation progress

Round	Open issues	Bugs	Vulns	Smells	Focus
1	447	13	2	432	Baseline analysis
2	202	0	0	202	Security + bugs + hygiene
3	40	0	0	40	Readonly props, JSX spacing, CSS, React keys
4	4	0	0	4	Nested ternaries, complexity, a11y
5	1	0	0	1	Regex / `RegExp.exec`
6	1	0	0	1	Regex rewrite → complexity trade-off
7	**0**	**0**	**0**	**0**	Complexity helpers — **cleared**

Net reduction: 447 → 0 (**100% of tracked issues closed**).

---

5. What was fixed (by theme)

Security & reliability

- Replaced weak PRNG usage in mail headers with `crypto.randomBytes`
- API Docker image runs as non-root user (`USER nestjs`)
- Stable string sorts (`localeCompare`); API bootstrap error handling
- Navbar overlays converted to native `<button>` elements for a11y / correctness

Maintainability & React quality

- Component props marked `Readonly<>` across sections/atoms
- Nested ternaries extracted to helpers / if-else
- Cognitive complexity reduced by splitting large render helpers

- Stable React keys (avoid array index where flagged)
- Context provider values memoized with `useMemo`
- JSX adjacent-text spacing made explicit where required

Accessibility & CSS

- Prefer native elements over ARIA roles (`button` , `img`) where flagged
- Removed empty / commented-out CSS blocks
- Contrast adjustment on an impact-studies fallback background

API / tooling hygiene

- Prefer `node:` protocol imports
- FormData helpers with safe stringification
- SMTP / Turbo env wiring for mail (operational; validated separately)

Content helpers

- Markdown/HTML content helpers rewritten to avoid catastrophic regex backtracking and to keep cognitive complexity within limits

---

6. Final metrics (Round 7)

Metric	Value
Bugs	**0**
Vulnerabilities	**0**
Code smells	**0**
Security hotspots	**0**
Open issues	**0**
Coverage	0.0% *(not instrumented)*
Duplicated lines density	~14.8%
Reliability / Security / Maintainability	**1.0 / 1.0 / 1.0** (A)

Quality gate conditions (observed)

Condition	Status	Notes
New violations	**OK**	0 new violations
New duplicated lines density	**OK**	Under threshold
New coverage	**ERROR**	0% vs required 80%

---

7. Recommendations for the team

1. **Coverage** — Add unit/integration tests (Jest / Vitest / Playwright as appropriate) and wire a coverage report into Sonar so the quality gate can pass on new code.
2. **Duplication (~15%)** — Optionally tackle high-duplication hotspots in shared section patterns; not blocking on issues, but useful for long-term maintainability.
3. **CI integration** — Run Sonar Scanner in CI on PRs against a shared Sonar server (or SonarCloud) so regressions are caught before merge.
4. **Keep the Navbar exception documented** — Revisit if active-link styling changes.

---

8. Artifacts & references

Artifact	Path
This final report (Markdown)	`docs/sonar-final-static-analysis-report.md`
This final report (PDF)	`docs/sonar-final-static-analysis-report.pdf`
Round 1 baseline	`docs/sonar-full-analysis-report.md`
Round reports 2–7	`docs/sonar-round{2–7}-analysis-report.md`
Round 7 raw export	`docs/sonar-round7-scan-raw.json`
Scanner config	`sonar-project.properties`

Local dashboard (during analysis): `http://localhost:9000/dashboard?id=eforte-new`

---

9. Sign-off

Item	Status
Bugs cleared	Yes
Vulnerabilities cleared	Yes
Code smells cleared	Yes
Secrets findings	None detected
Coverage gate	Not met (0% — follow-up)

Conclusion: The project is **clean of SonarQube bugs, vulnerabilities, and code smells**. Remaining quality-gate failure is limited to **missing test coverage configuration**, which should be tracked as a separate engineering follow-up.

Prepared for internal team sharing — eForte New static analysis engagement, October 2026.